

Project's Report

Andrea Frassinetti ID: 256223
Alessandro Pais Bianco ID: 258030

1 Intruduction and Objective

This report presents the development and evaluation of Project B, titled "Learning a Terminal Constraint Set". The core objective of this project is to approximate a control-invariant set using a neural network and to utilize this approximation as a terminal constraint within a Model Predictive Control (MPC) framework, in order to ensure recursive feasibility.

The proposed approach addresses this by learning the N -step backward reachable set of an equilibrium set $\mathcal{S}$.

The following sections will detail how we generate the data, the neural network architecture, and a comparative analysis demonstrating how this terminal constraint maintains feasibility in scenarios where a standard MPC with a short horizon would otherwise fail.

2 Problem Definitions

Ensuring **recursive feasibility** is a critical challenge when applying Model Predictive Control (MPC) to non-linear systems like double pendulum. In standard finite-horizon MPC, the controller may guide the system into states that, while satisfying constraints in the short term, leave no possible control sequence to prevent future constraint violations or to return the system to a stable equilibrium. A robust theoretical solution involves constraining the state at the end of the prediction horizon to lie within a control-invariant set. For complex robotic systems, such as the single and double pendulums considered here, characterizing these sets analytically is often non-trivial.

2.1 The Role of Terminal Constraints

To guarantee feasibility, control theory suggests the inclusion of a terminal constraint, requiring the predicted state at the end of the horizon to belong to a safe set: $x_N \in \mathcal{X}_f$. Typically, this set is restricted to the equilibrium manifold $\mathcal{S}$, where the joint velocities are zero ($\dot{q} = 0$). However, enforcing a zero-velocity terminal constraint is often **overly conservative**. If the prediction horizon N is too short, the robot might be physically unable to decelerate to a full stop within the time, rendering the optimization problem infeasible even if a safe path exists over a longer duration.

2.2 The N -step Backward Reachable Set (BRS)

To expand the region of operation without sacrificing safety, we define the **N -step Backward Reachable Set** ($\mathcal{BRS}_N$). This set is formed by all initial states x_{init} from which there exists at least one admissible control sequence capable of driving the system into the equilibrium set $\mathcal{S}$ within N discrete time steps.

The membership of a state x_{init} in this set is determined by solving a feasibility-based Optimal Control Problem (OCP):

$$\begin{aligned}
& \min_{X,U} && 1 \\
& \text{s.t.} && x_{i+1} = f(x_i, u_i), && i = 0, \dots, N-1 \\
& && u_{min} \leq u_i \leq u_{max}, && i = 0, \dots, N-1 \\
& && q_{min} \leq q_i \leq q_{max}, && i = 0, \dots, N-1 \\
& && \dot{q}_{min} \leq \dot{q}_i \leq \dot{q}_{max}, && i = 0, \dots, N-1 \\
& && x_0 = x_{init} \\
& && \dot{q}_N = 0 \quad (\text{Terminal Equilibrium})
\end{aligned} \tag{1}$$

If a feasible solution exists, the state is labeled as **Safe** (1); otherwise, it is **Unsafe** (0).

2.3 Physical Limits and Feasibility Challenges

The definition of this problem is strictly tied to the **physical actuators limits**. The presence of maximum torque (τ_{max}) and joint velocity limits (v_{max}) creates regions in the state space where the double pendulum's momentum becomes "unrecoverable." For instance, if the second link reaches a high angular velocity while far from the vertical equilibrium, the available torque may not be sufficient to counteract gravity and inertia within the prediction horizon. Properly modeling these boundaries is crucial, as they delineate the "point of no return", states from which a crash is inevitable regardless of the future control actions.

2.4 Data-Driven Safety Barrier via Neural Networks

Since solving the OCP above in real-time is computationally expensive, we approximate the indicator function of the $\mathcal{BRS}_N$ using a **Neural Network (NN)**. The reasoning follows a four-step pipeline:

1. **Sampling:** a large set of states are sampled across the robot's state space, ensuring coverage of both critical boundaries and safe regions.
2. **Labeling:** The OCP is solved for each sample to verify feasibility.
3. **Learning:** A classifier is trained to learn the boundary between safe and unsafe regions.
4. **Deployment:** The trained NN is embedded into the MPC as a symbolic constraint using `14casadi`.

This neural constraint acts as a **dynamic safety barrier**. By requiring the terminal state of the MPC prediction to satisfy $NN(x_N) \geq 0.5$, the controller is preemptively restricted from guiding the system into "unrecoverable" high-velocity configurations. This effectively forces the robot to "brake" in advance, maintaining recursive feasibility even with a short prediction horizon.

3 Data Generation and Labelling

To create the training dataset, we iteratively sampled 5000 states, labeling each as Safe (1)

or Unsafe (0) based on OCP feasibility. Four distinct datasets were generated with varying horizons $N \in \{25, 50, 100, 200\}$ to analyze how the safe set topology evolves. We introduced environmental constraints acting as a "virtual wall": the first link is physically barred from crossing the vertical axis (q_{min}), and the elbow joint is restricted to a realistic range. These hard constraints create a sharp, boundary between safe and unsafe zones, providing a rigorous test for the neural network. Instead of sampling uniformly across the entire state space, we focused on a region of interest near the critical constraints to maximize the information gain during training. The sampling bounds were defined as:

$$\begin{aligned} -3.5 &\leq q_1 \leq 1.5, & -0.5 &\leq q_2 \leq 1.5 \\ -6.0 &\leq \dot{q}_1 \leq 6.0, & -6.0 &\leq \dot{q}_2 \leq 6.0 \end{aligned} \tag{2}$$

The torque limits were derived considering the worst-case static scenario: both links extended horizontally with masses modeled as concentrated parameters at the tips. Initially, a scaling factor < 1 was applied to enforce strict underactuation. However, this yielded an overly sparse dataset (few safe points). Consequently, the limit was increased to a factor of 1.1 to expand the feasible region and ensure sufficient positive samples. Finally, to enhance solver performance, we implemented a linear-decay warm start. Instead of a random initialization, the solver is provided with an initial guess trajectory where velocities decrease linearly to zero. This significantly improved convergence rates and the discovery of feasible states, as visualized in Figure 1.

4 Neural Network's Architecture and Training

After generating the dataset we trained the **Neural Network** to classify if the states lies or not in the $\mathcal{BRS}_N$.

4.1 Network Architecture

To ensure a balance between approximation accuracy and computational efficiency within the MPC loop, we designed the following structure:

- **Input:** 4 which are the states for the double pendulum ($q_1, q_2, \dot{q}_1, \dot{q}_2$)
- **Hidden size:** 32 beacuse the problem is quite easy (computationally speaking) so use a larger structure is not so different (it was tried also 64 but the improvement was not relevant)
- **Output:** 1 neuron that gives a scalar value that represent where the state is or not into the set

4.2 Training Setup

The model was implemented in PyTorch and trained using the generated datasets. The hyper-parameters used for the training phase are summarized in Table 1.

We used the BCE loss function due to the fact that is suited for binary classification task as our. It is combined with the sigmoid activation function so the training process is simplified. The role of sigmoid is to transform the raw logits dropped by the BCE in a probability value between 0 and 1.

Hyper-parameter	Value
Loss Function	Binary Cross-Entropy (BCE)
Optimizer	Adam
Learning Rate	1×10^{-3}
Batch Size	32
Epochs	200
Hidden Units	32
Epochs	200
Activation (Hidden)	Tanh
Activation (Output)	Sigmoid

Table 1: Hyper-parameters of the neural network

4.3 Validation of the Learned Boundary

To evaluate the quality of the learned constraint, we first performed a **qualitative assessment** by visualizing the decision boundary. Although the state space is 4-dimensional, Figure 2 projects the network’s output onto the 2D position plane (q_1, q_2) . The heatmap clearly distinguishes the **Safe region** (Green, where $NN(x) \geq 0.5$) from the **Unsafe region** (Red). Notably, the decision boundary aligns perfectly with the ”Virtual Wall” constraint defined in Section 3, confirming that the network has correctly encoded the physical limits of the workspace.

Complementing this visual inspection, we conducted a **quantitative validation** to test the network’s generalization capabilities. For each horizon model ($N \in \{25, 50, 100, 200\}$), we generated a validation set of 200 unseen random states. We solved the OCP for each state to obtain the ground truth and compared it against the network’s prediction.

The results are summarized in Table 2. The network achieves high classification performance across all horizons. While there is a slight drop in accuracy on the unseen validation set compared to the internal test set for longer horizons ($N \geq 50$), the accuracy consistently remains **above 90%**. This confirms that the model generalizes well and provides a reliable safety barrier for the MPC.

Table 2: Neural Network Accuracy: Training vs. Validation (200 random samples)

Horizon (N)	Training Acc. (%)	Validation Acc. (%)
25	95.6	95.5
50	96.8	91.5
100	98.0	90.5
200	97.3	90.5

5 MPC Implementation

The aim at this point is to use the $\mathcal{BRS}_N$ obtained by the neural network to constrain the final state in this MPC problem:

$$\begin{aligned}
& \min_{X,U} \quad \sum l(x_k, u_k) \\
& \text{s.t.} \quad x_{k+1} = f(x_k, u_k), \quad k = 0, \dots, N-1 \\
& \quad \quad x_k \in \mathcal{X}, \quad k = 0, \dots, N-1 \\
& \quad \quad u_k \in \mathcal{U}, \quad k = 0, \dots, N-1 \\
& \quad \quad x_0 = x_{init} \\
& \quad \quad g(x_N) \geq 0.5
\end{aligned} \tag{3}$$

- N : Time horizon for the simulation
- $l(x_k, u_k)$: Cost at instant k
- $\mathcal{X}, \mathcal{U}$: State control and the input control. they must be the same of the one used for the OCP
- $g(x_N)$: Is the activation function that provide us the likelihood on how probably the state is or not feasible. It setted over 0.5 because over that the NN think it's more probable that is a feasible state.

For what concern the Torque we scale that value in order to compensate some approximation error of the solver. For some reason in fact using the same value of the torque limit of the OCP the MPC does not work as we expected.

6 Discussion

6.1 Experimental Setup and Tuning

To evaluate the effectiveness of the learned terminal constraint, the MPC was initialized with the same physical parameters and constraints used in the Data Generation phase (OCP). The initial states x_{init} for the simulation were generated randomly but filtered: a state is accepted only if the neural network classifies it as feasible ($NN(x_{init}) \geq 0.5$).

However, initial tests revealed a discrepancy between the offline OCP and the online MPC. While the OCP solver could find feasible trajectories with a torque scaling factor of 1.1, the MPC solver frequently failed to converge under identical strict limits. This is likely attributed to:

1. **Discretization Errors:** The accumulation of numerical errors over the MPC horizon.
2. **Solver Tolerances:** The local optimization in MPC (IPOPT) is more sensitive to non-convexities introduced by the neural network gradient than the offline OCP.

To achieve convergence and demonstrate the braking capability of the system, we relaxed the actuation constraints by increasing the torque scaling factor from **1.1 to 2.0**. While this deviates from the strict physical limits used in training, it allows us to verify the logical behavior of the safety barrier: provided sufficient actuation, the controller correctly interprets the learned boundary.

6.2 Analysis of System Dynamics

With the relaxed torque limits, the MPC successfully stabilizes the system. Figures 3,4,5,6 illustrates the evolution of states and inputs:

- **Positions:** Despite the aggressive maneuvers, the trajectories remain smooth. Crucially, the system strictly respects the geometric constraints, including the "Virtual Wall" along the negative vertical axis. This demonstrates that the neural network provides a continuous and well-defined safety boundary.
- **Velocities:** The velocity profiles exhibit a characteristic "braking" behavior. As the system approaches the limits of the learned safe set (where $NN(x)$ drops), velocities are sharply reduced. This confirms that the terminal constraint $NN(x_N) \geq 0.5$ acts as a preventative measure, forcing the robot to decelerate well before hitting the wall. In all successful simulations, velocities converge to zero ($\dot{q} \rightarrow 0$).
- **Torques:** In the initial phase, the torque often saturates at the maximum limit (scaling 2.0). This indicates the controller is applying maximum effort to correct the trajectory and remain within the viability kernel. Joint 1 generally requires higher torque than Joint 2 to support the structure's mass. Once equilibrium is approached, control inputs stabilize to gravity-compensation values.

6.3 Critical Discussion and Limitations

While the system behaves safely, two anomalies were observed during extensive testing which suggest potential avenues for future investigation:

1. Impact of the Terminal Constraint: We performed an ablation study by disabling the neural terminal constraint (flagging it to 0). Unexpectedly, the standard MPC successfully stabilized the system in the majority of cases, with only a marginal increase in failure rate. This implies that for the sampled initial states, the prediction horizon might already be sufficient to avoid "dead ends" without an explicit terminal set, or that the generated initial states were not sufficiently "critical" to highlight the necessity of the BRS.

2. Invariance to Horizon Length: Comparing the behavior of models trained with different horizons ($N \in \{25, 50, 100, 200\}$), we observed no substantial difference in the controller's performance or success rate. Theoretically, a shorter horizon model ($N = 25$) should be more conservative (smaller safe set) than a longer one ($N = 200$). The lack of behavioral difference, combined with the high validation accuracy of all networks (Section 4), suggests that the bottleneck may lie in the MPC formulation itself or that the "Virtual Wall" constraint dominates the safety definition more than the dynamic limitations.

7 Conclusion

This project successfully implemented a pipeline to approximate the N -step Backward Reachable Set of a double pendulum using Neural Networks and integrate it into an MPC framework via `14casadi`. The neural network demonstrated high accuracy ($> 90\%$) in distinguishing safe states from inevitable collisions. Qualitatively, the controller respects the learned boundaries and performs the expected braking maneuvers. However, the need for relaxed torque limits in the online phase and the limited impact of the constraint on success rates highlight the

challenges of transferring offline-learned boundaries to real-time solvers. Future work should focus on robustifying the MPC formulation against numerical discrepancies and testing on more aggressive initial states.

8 Plots

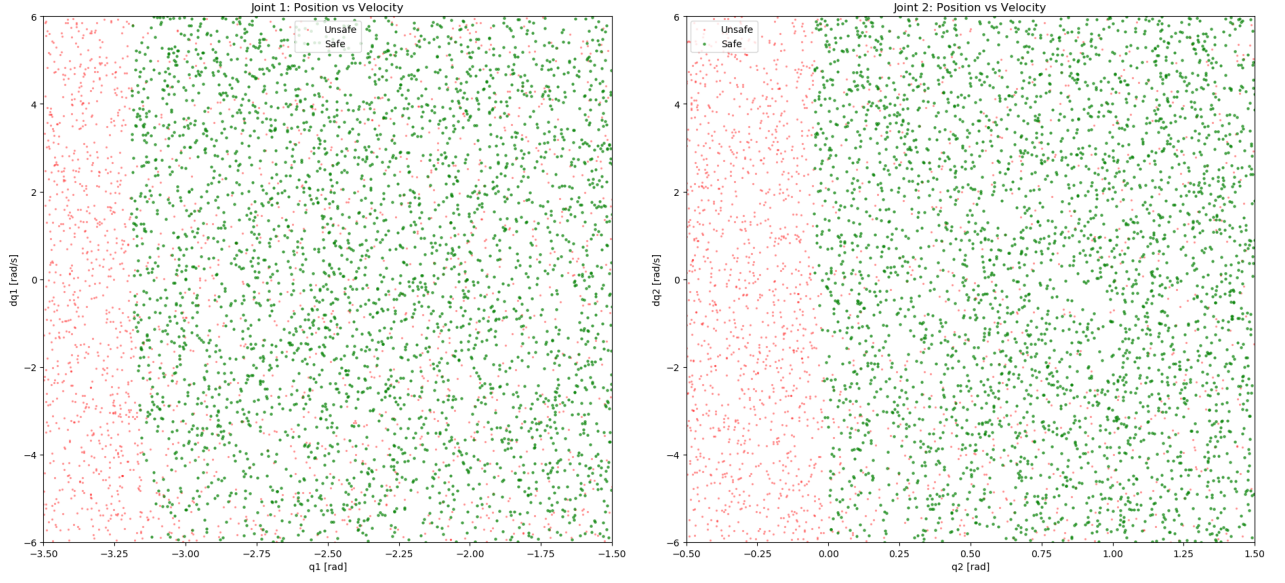
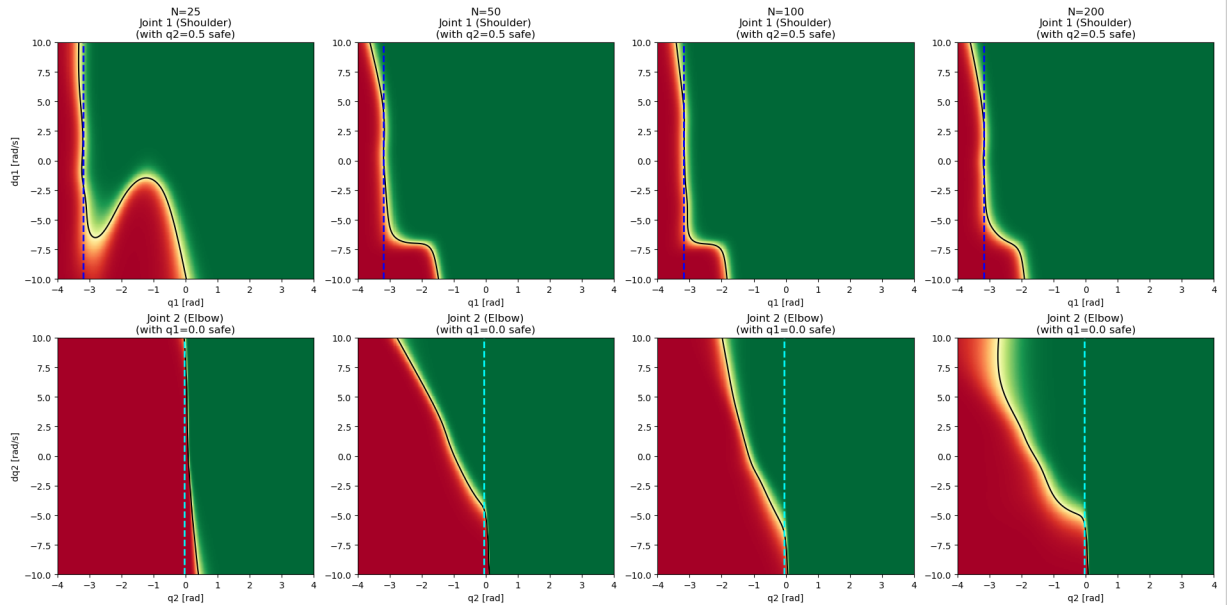
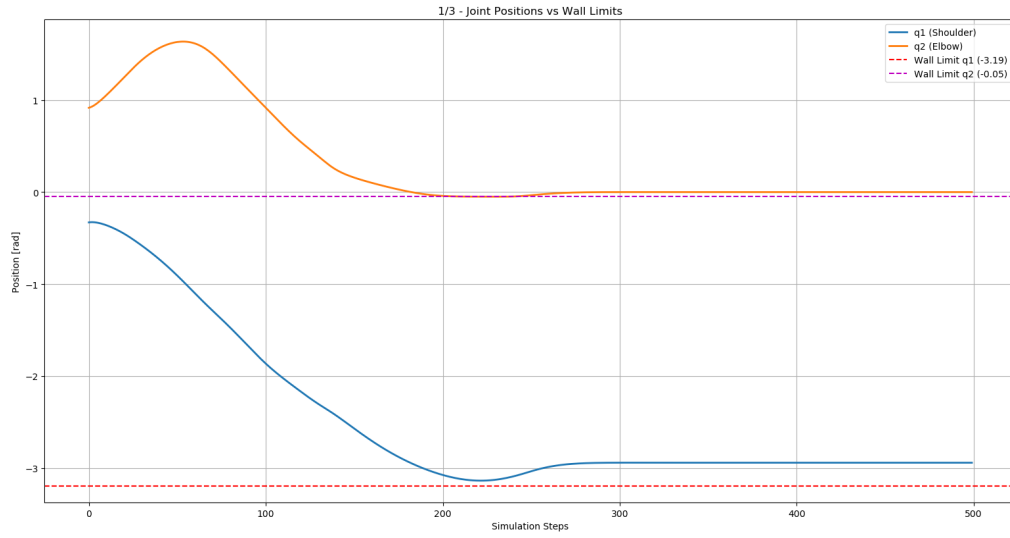
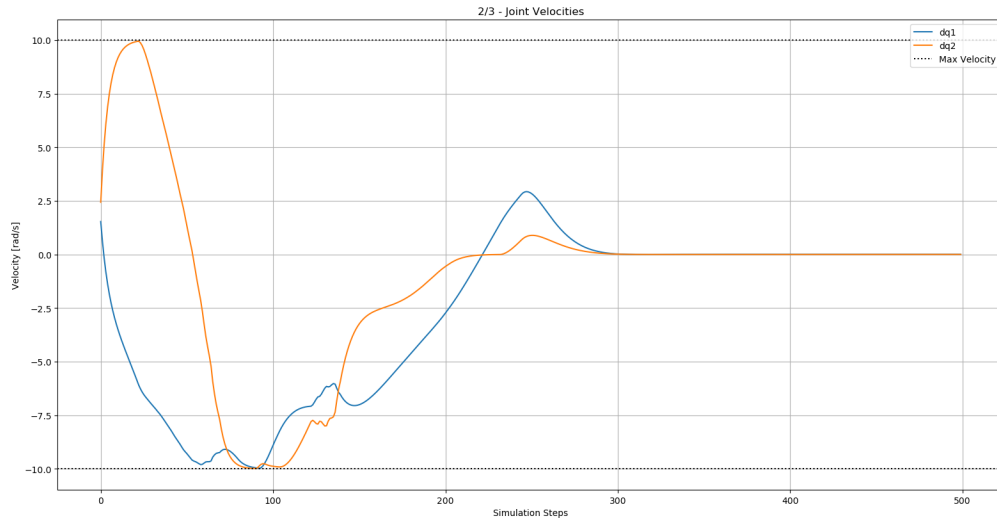


Figure 1

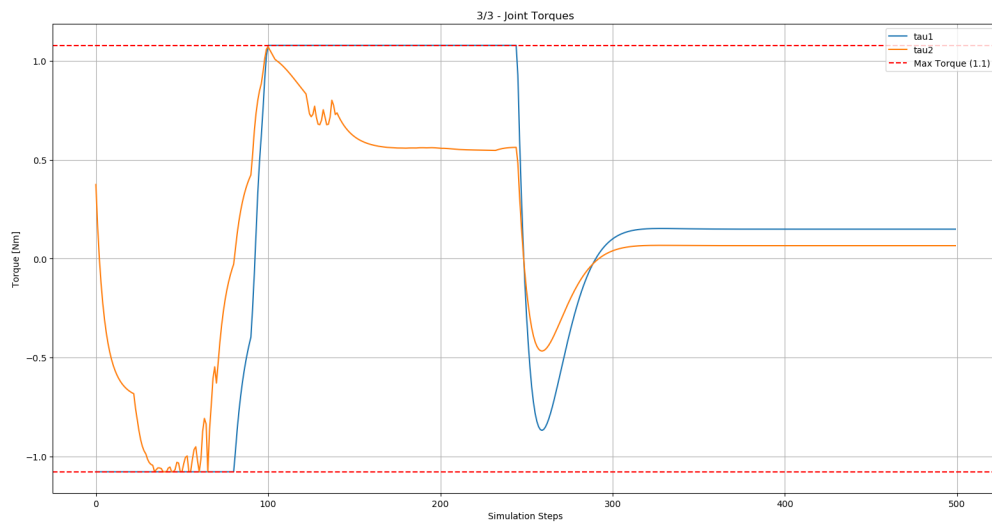
Figure 2: Comparison of the neural network model with horizon $N=25, 50, 100, 200$



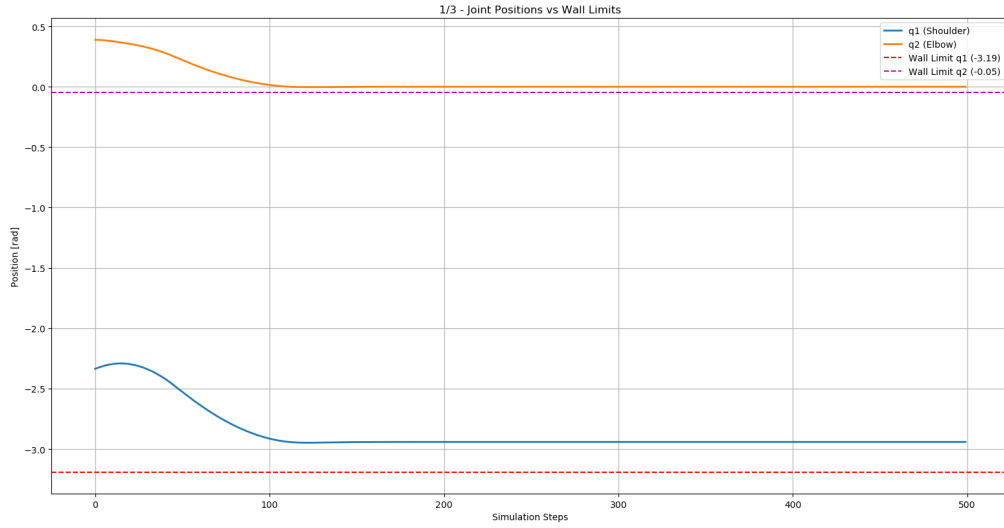
(a) Joint Position



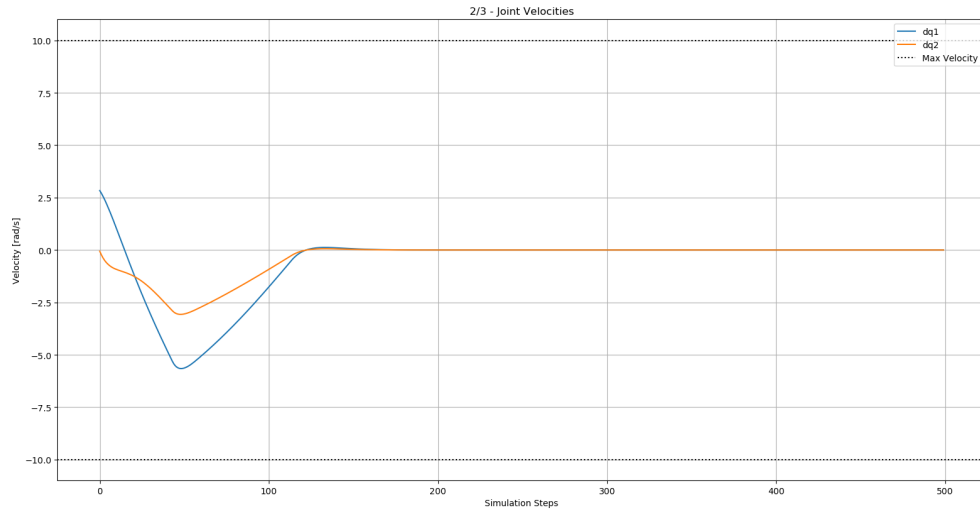
(b) Joint Velocity



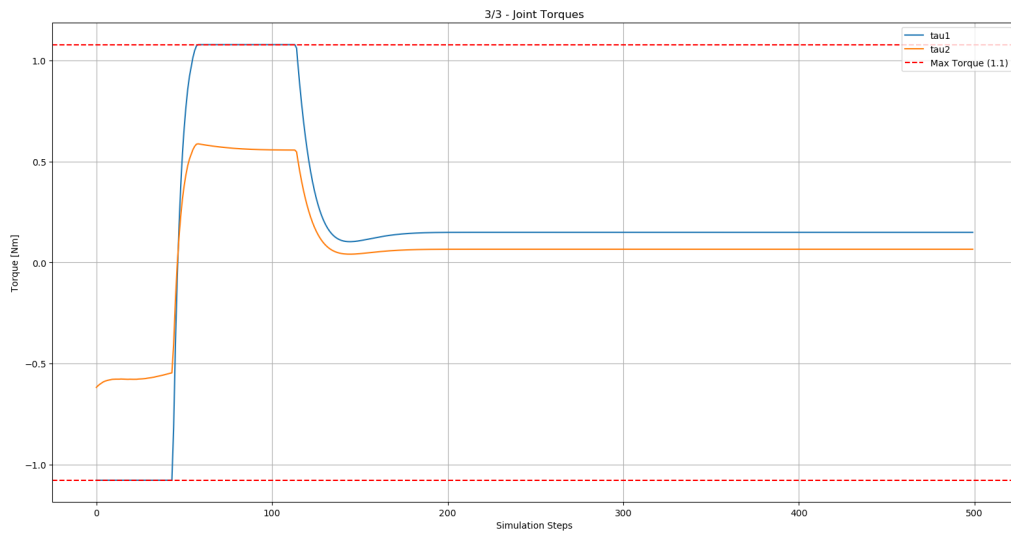
(c) Joint Torque



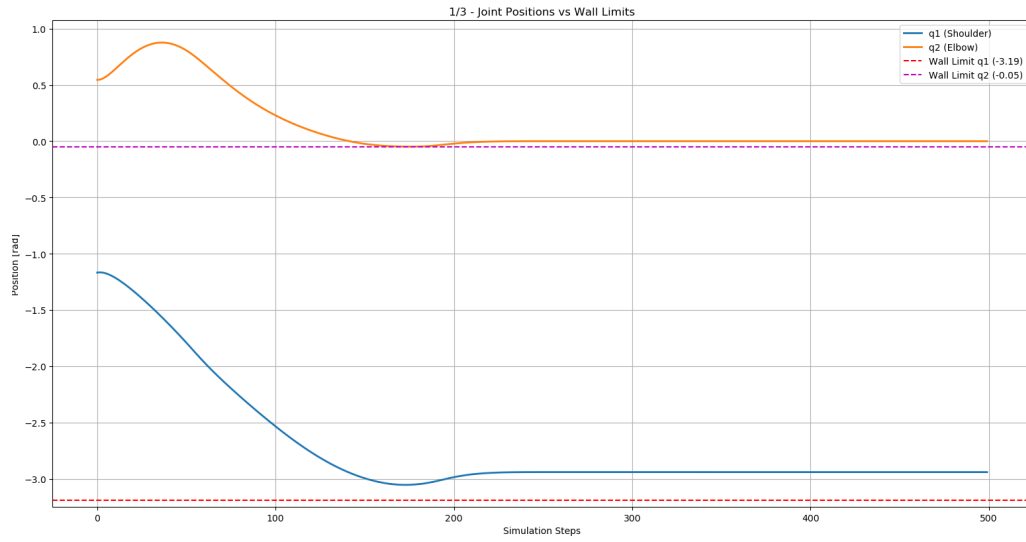
(a) Joint Position



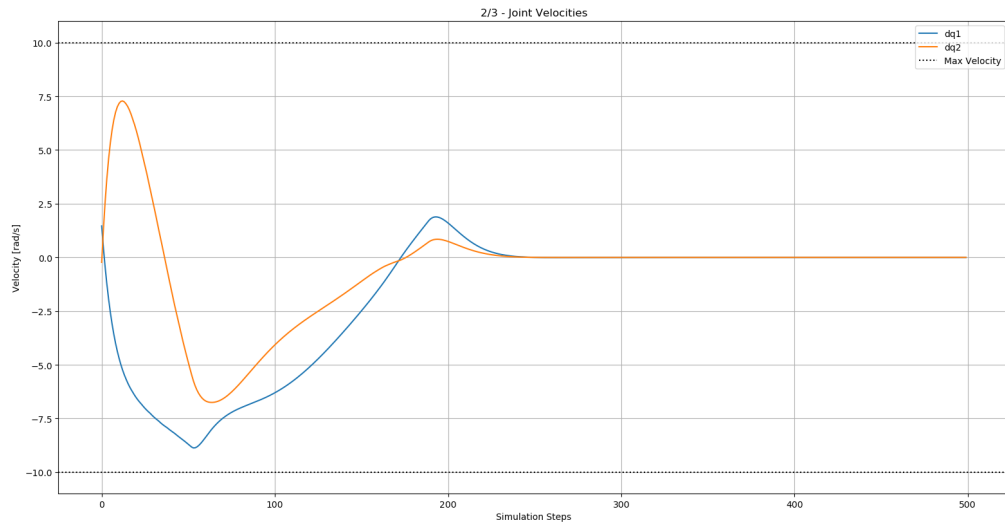
(b) Joint Velocity



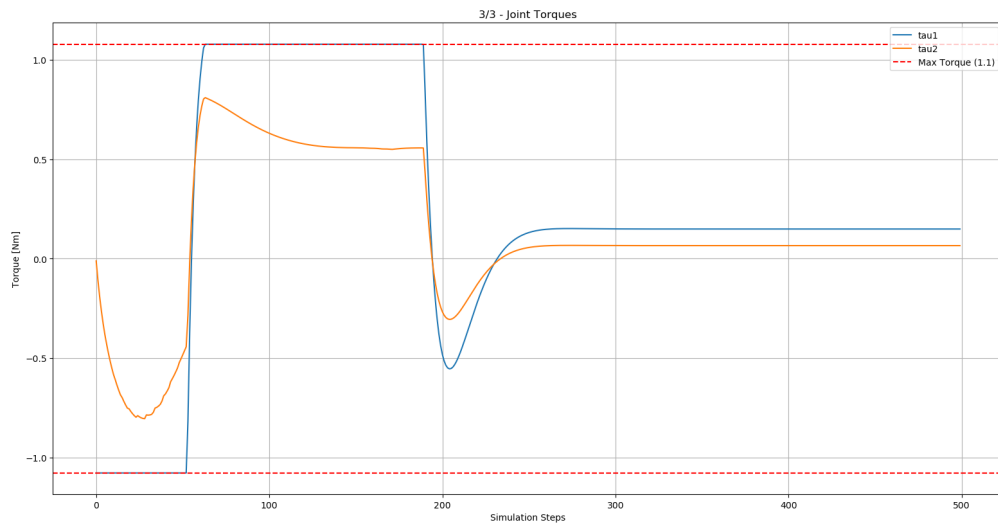
(c) Joint Torque



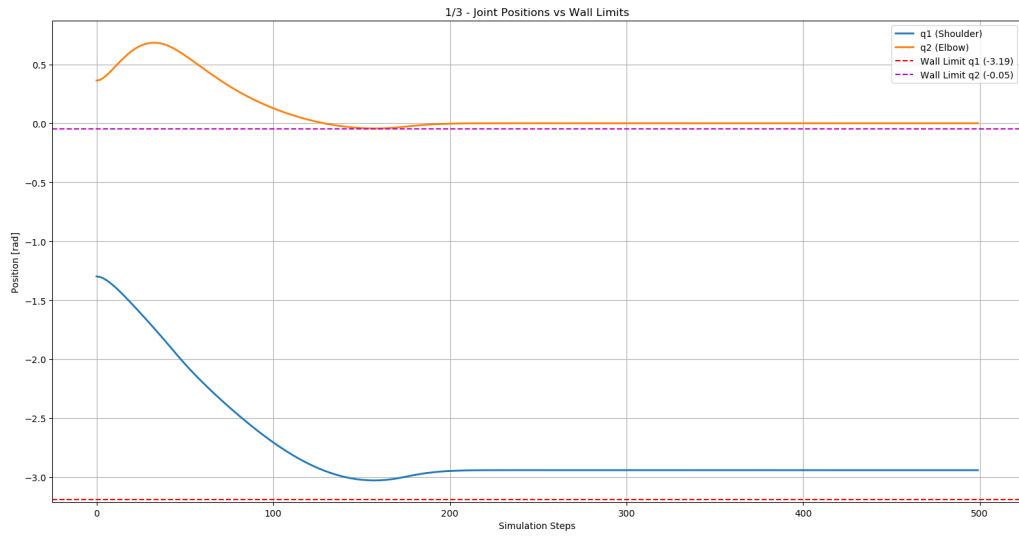
(a) Joint Position



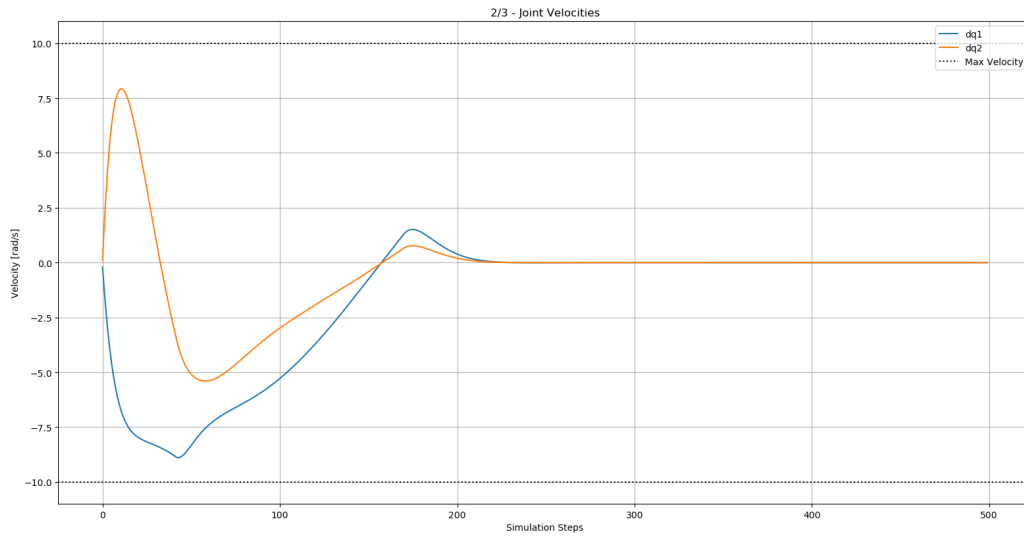
(b) Joint Velocity



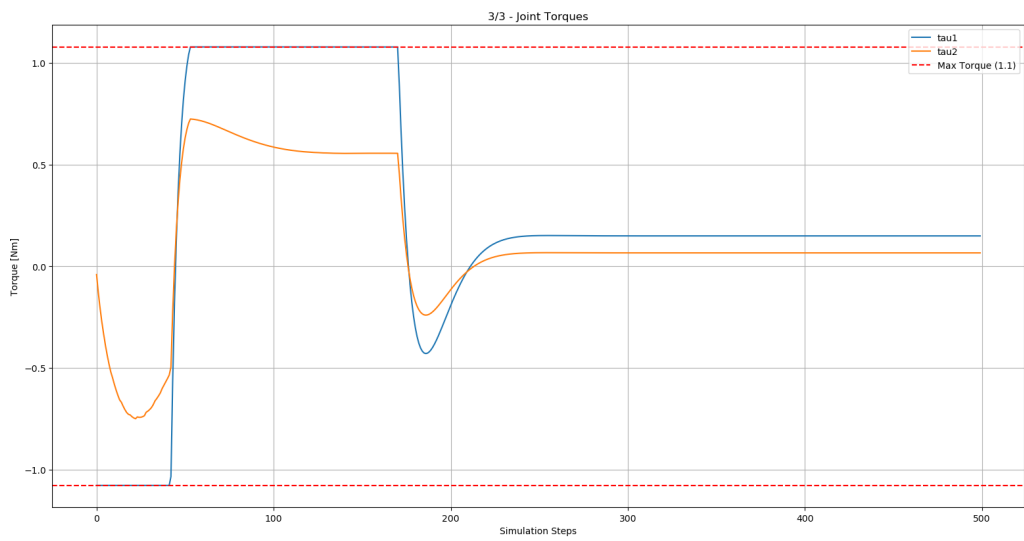
(c) Joint Torque



(a) Joint Position



(b) Joint Velocity



(c) Joint Torque

Figure 6: N=200